# App-Level TinyML for Smartphone Battery-Life Prediction

Keno Schürger
Technische Hochschule Würzburg-Schweinfurt (THWS)
Email: keno.schuerger@study.thws.de

## I. Introduction

Predicting the remaining battery life of a smartphone is a long-standing problem with two qualitatively different solution classes. *Analytical* methods (e.g. the linear extrapolation used by most legacy "Settings → Battery" screens) project the recently observed drain rate into the future. *Machine-learning* methods, in contrast, attempt to exploit context: which apps are active, how bright is the screen, what is the battery temperature.

Since Android 12 (October 2021), the platform exposes a system estimator via `PowerManager.getBatteryDischargePrediction()` [1], which returns a duration estimate computed inside the operating system. As a system process the estimator runs at a privilege level not available to third-party apps and can in principle access hardware counters that the Android permission model does not expose to user-installed applications. The companion method `isBatteryDischargePredictionPersonalized()` indicates whether the returned estimate has been adapted to the specific device's usage history.

We ask: *how well can a TinyML model running inside an Android app predict remaining battery life, compared to analytical baselines and Google's native API — measured on accuracy and on-device efficiency?* TinyML here refers to small quantised neural networks (typically tens of kilobytes) designed for low-power edge devices [2]. While the TinyML literature largely targets microcontrollers, smartphones share two relevant properties: deployment constraints (model size matters for APK weight, latency matters for battery cost) and ubiquity.

This paper provides an empirical answer for the smartphone case. We report a six-way head-to-head comparison on identical real-world data ($X$ measurements collected over an $X$-day window from four Android devices spanning two manufacturers), explicitly controlled for a class of data-leakage pitfalls that we found to inflate apparent results by a data-diversity-dependent factor.

*Contributions*

1) **Methodological observation:** On sliding-window time-series data, a random shuffle of *sequences* (the default `train_test_split`) silently leaks future information into training. On our data the inflation factor depends on data diversity ($\sim X\times$ on the multi-device dataset, $\sim Y\times$ on the single-device subset). We propose and measure a segment-level split as the principled alternative.
2) **Multi-device six-way comparison with statistical rigour:** We collect data on four Android devices (Xiaomi 2107113SG, Pixel 7 Pro, 8 Pro and 9 Pro XL) and compare TinyML against four explicit baselines (Random Forest, mean predictor, linear drain rate, exponential fit) and the native Google API. Differences are reported with bootstrap 95% confidence intervals; pairwise significance uses a paired permutation test for MAE and a paired bootstrap on $\Delta C$ for the $C$-index (per-sample swap is invalid for a pairwise metric), with Benjamini–Hochberg correction over 15 tests per family.
3) **Device-dependent learnability:** Per-device evaluation reveals a strong hardware effect. On the Pixel 7 Pro the TinyML model achieves $C{=}X$, several times the gap to the mean predictor that the same model attains on the Xiaomi ($C{=}Y$). This demonstrates that the signal available to an app-level model is dominated by the underlying sensor stack, not by the model architecture or training budget.
4) **Model-independent feature diagnostics:** Beyond permutation-importance on the Random Forest, we report Pearson, Spearman and mutual-information statistics for all ten features. This separates a model-specific failure ("Conv1D is the wrong architecture") from a data-level limitation ("the publicly available features only carry enough signal on certain hardware").
5) **Reproducible open pipeline.** All code, configuration, raw data, intermediate artefacts, and the auto-generated report are released in a structured repository. A single command (`python run_pipeline.py`) reproduces every number in this paper.

## References

[1] Android Open Source Project, "PowerManager.getBatteryDischargePrediction()," 2021, android API level 31 (Android 12). [Online]. Available: https://developer.android.com/reference/android/os/PowerManager#getBatteryDischargePrediction()

[2] C. Banbury, V. J. Reddi, P. Torelli, J. Holleman, N. Jeffries, C. Kiraly, P. Montino, D. Kanter, S. Ahmed, D. Pau *et al.*, "MLPerf Tiny benchmark," in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021. [Online]. Available: https://arxiv.org/abs/2106.07597